

DataFrames, Filtering, Missing Data, Joins, and Grouping

DAEN 300

Alexander Abuabara

Fall 2026

Learning objectives

By the end, I hope you can:

- ▶ inspect a DataFrame and its schema;
- ▶ select rows and columns explicitly;
- ▶ detect and resolve missing values;
- ▶ join related tables with validation;
- ▶ aggregate data at a declared grain.

Pipeline question (for instance)

Which customer segment generated the most completed-order revenue?

`orders + customers`



`segment_summary`

DataFrame: a labeled, typed table

```
import pandas as pd

orders = pd.DataFrame({
    "order_id": [101, 102, 103, 104],
    "customer_id": [1, 2, 1, 3],
    "amount": [120.0, None, 75.0, 210.0],
    "status": ["complete", "pending",
               "complete", "complete"]
})
```

- ▶ Rows are observations.
- ▶ Columns are variables with a dtype (stands for data type).
- ▶ The index labels rows; it is not automatically a "business" key.

First checks

```
orders.shape  orders.head()
orders.dtypes orders.info()
```

Filtering expresses a row predicate

```
mask = (orders["status"] == "complete") & (orders["amount"] >= 100)

high_value = orders.loc[mask, ["order_id", "customer_id", "amount"]]
```

SQL analogy

```
.loc[row condition, columns]
corresponds to
SELECT columns WHERE condition
```

- ▶ Use `&`, `|`, and `~` for elementwise logic.
- ▶ Parenthesize every comparison.
- ▶ Use `.iloc` for positions, `.loc` for labels and masks.

```
df.iloc[0] # first row
```

```
df.iloc[0, 1] # first row, second column
```

```
df.loc[df["sales"] >= 100, ["city", "sales"]]
```

Missing data is part of the schema

```
# Profile nulls
orders.isna().sum()

# Choose a rule tied to meaning
clean = (
    orders.dropna(subset=["customer_id"])
    .copy()
)
clean["amount"] = clean["amount"].fillna(0)

# Nullable integer when NA is valid
clean["customer_id"] = (
    clean["customer_id"].astype("Int64")
)
```

Ask before filling

Does missing amount mean zero, unknown, not applicable, or a pipeline defect?

- ▶ Detect: `isna()`, `notna()`
- ▶ Remove: `dropna()`
- ▶ Replace: `fillna()`
- ▶ Preserve: nullable dtypes such as `Int64`

fillna()?

fillna() replaces missing values with the value or rule that **you specify**.

```
# Fill with zero
df["amount"] = (
    df["amount"].fillna(0)
)

# Fill with the median
df["amount"] = (
    df["amount"].fillna(
        df["amount"].median()
    )
)

# Fill text values
df["city"] = (
    df["city"].fillna("Unknown")
)
```

Different value per column

```
df = df.fillna({
    "amount": 0,
    "city": "Unknown"
})
```

Choose based on meaning

Filling an unknown amount with zero states that no amount occurred. Use zero only when that interpretation is valid.

Joins combine tables at a key

```
customers = pd.DataFrame({
    "customer_id": [1, 2, 3],
    "segment": ["SMB", "Enterprise", "SMB"]
})

enriched = orders.merge(
    customers,
    on="customer_id",
    how="left",
    validate="many_to_one",
    indicator=True
)
```

how	keeps
inner	matching keys
left	every left row
right	every right row
outer	all keys

Guardrails

`validate=` checks key cardinality.
`indicator=True` exposes unmatched rows.

Checking joins

validate: check key cardinality

```
enriched = orders.merge(
    customers,
    on="customer_id",
    how="left",
    validate="many_to_one",
    indicator=True
)
```

`many_to_one` requires:

- ▶ repeated keys allowed in orders;
- ▶ unique keys required in customers;
- ▶ invalid cardinality raises an error.

indicator: inspect matches

<code>_merge</code>	Meaning
both	matched key
left_only	no customer match
right_only	no order match

```
enriched["_merge"].value_counts()

unmatched = enriched.loc[
    enriched["_merge"].eq("left_only")
]
```

Difference: `validate` checks the expected relationship; `indicator` reveals unmatched keys.

Grouping changes the grain

```
segment_summary = (  
    enriched.loc[enriched["status"].eq("complete")]  
    .groupby("segment", as_index=False)  
    .agg(  
        orders=("order_id", "nunique"),  
        revenue=("amount", "sum"),  
        avg_order=("amount", "mean")  
    )  
    .sort_values("revenue", ascending=False)  
)
```

Mental model

Split rows by key, apply an aggregation, combine one result per group.

Grain check

Input: one row per order.
Output: one row per segment.

The pipeline as relational operations

Intent	pandas	Relational idea
Choose rows	Boolean mask + <code>.loc</code>	selection / WHERE
Choose columns	<code>df[[...]]</code>	projection / SELECT
Add attributes	<code>merge(...)</code>	join
Summarize grain	<code>groupby().agg()</code>	grouping + aggregation
Handle nulls	<code>isna, dropna, fillna</code>	null semantics

Grain and row-count checks

After every operation, ask: What is the row grain? Which keys should be unique? How many rows should survive?

Using assert for data-quality checks

`assert` verifies that an expected condition is true. If the condition is false, Python raises an `AssertionError`.

Basic syntax

```
assert condition, "Error message"
```

Pandas examples

```
# Order IDs must be unique
assert orders["order_id"].is_unique

# Customer IDs cannot be missing
assert orders["customer_id"].notna().all()
```

Checks after a join

```
# A left join preserves order rows
assert len(enriched) == len(orders)

# Every order matched a customer
assert enriched["_merge"].eq(
    "both"
).all()
```

Reliability checks belong beside transformations

```
# Contract checks
assert orders["order_id"].is_unique
assert orders["customer_id"].notna().all()

# Join checks
assert enriched["_merge"].eq("both").all()
assert len(enriched) == len(orders)

# Aggregation reconciliation
expected = enriched.loc[
    enriched["status"].eq("complete"), "amount"
].sum()
assert segment_summary["revenue"].sum() == expected
```

Data engineering habit

Pair each business transformation with a test for grain, completeness, or reconciliation.

Practice 1: Choose the pipeline

Question

Which pipeline calculates completed revenue by customer segment?

A

```
(orders.merge(customers)
        .groupby("segment")
        ["amount"].sum())
```

B

```
orders.loc[
    orders["status"].eq("complete")
].merge(
    customers,
    on="customer_id",
    validate="many_to_one"
).groupby("segment")[
    "amount"
].sum()
```

C

```
orders.groupby("customer_id")[
    "amount"
].sum()
```

Practice 1: Choose the pipeline

Question

Which pipeline calculates completed revenue by customer segment?

A

```
(orders.merge(customers)
         .groupby("segment")
         ["amount"].sum())
```

B

```
orders.loc[
    orders["status"].eq("complete")
].merge(
    customers,
    on="customer_id",
    validate="many_to_one"
).groupby("segment")[
    "amount"
].sum()
```

C

```
orders.groupby("customer_id")[
    "amount"
].sum()
```

Answer: B

It filters completed orders, joins the customer segment, and then aggregates revenue by segment.

Practice 2: Find the data-quality risk

The join produces more rows than the original orders table.

```
enriched = orders.merge(  
    customers,  
    on="customer_id",  
    how="left")
```

What is the most likely cause?

- A Some order amounts are missing.
- B Customer IDs are duplicated in customers.
- C Some orders have a pending status.

Practice 2: Find the data-quality risk

The join produces more rows than the original orders table.

```
enriched = orders.merge(  
    customers,  
    on="customer_id",  
    how="left")
```

What is the most likely cause?

- A Some order amounts are missing.
- B Customer IDs are duplicated in customers.
- C Some orders have a pending status.

Answer: B

Duplicate customer keys can duplicate order rows during the join.

```
# Prevent the problem  
orders.merge(  
    customers,  
    on="customer_id",  
    how="left",  
    validate="many_to_one")  
# Check that grain is preserved  
assert len(enriched) == len(orders)
```